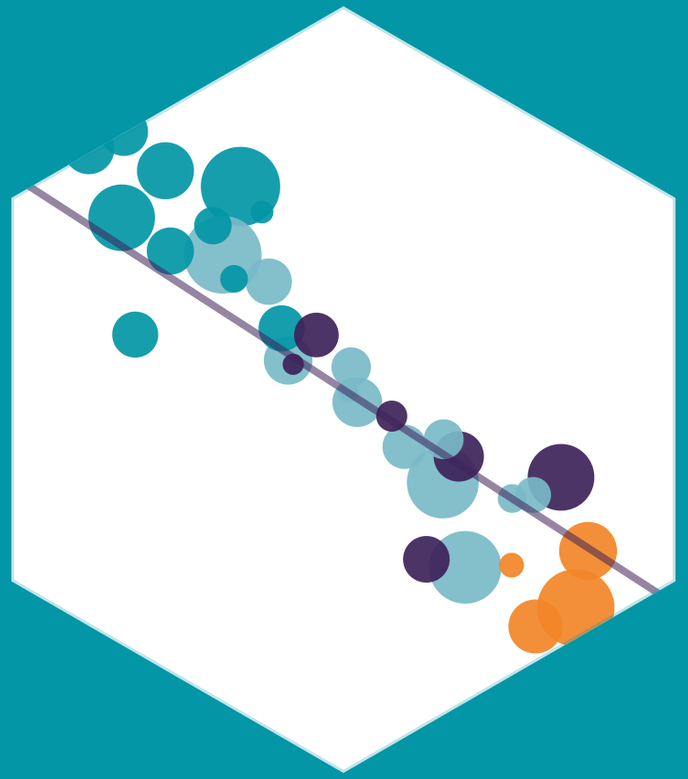
ECONOMETRICS

TA SESSION 4

Barcelona School of Economics
Lucia Sauer
lucia.sauer@bse.eu



Overview

- Introduction to Stata: the window, the toolbar, do-files
- The anatomy of a Stata command and its stored results
- Global hypothesis testing: is the model useful at all?
- Multiple hypothesis testing: are `job19` and `job20` jointly significant?

Why These Topics?

Stata

- The standard software in applied economics: most replication packages you will read are **do-files**

Global Hypothesis Testing

- The F value at the top of every regression output: what is it testing?

Multiple Hypothesis Testing

- Testing **several restrictions at once**: restricted vs unrestricted model

Introduction to Stata

What is Stata?

A **statistical software package** for data analysis, created in 1985 and still widely used in economics, political science and public health.

1

Paid software

Stata needs a **licence** (BE, SE or MP editions). Python and R are **free and open source**: anyone can install them, read their code and extend them.

2

The old standard

For years it was the tool in statistics and econometrics: new methods from papers were released as a **Stata package** first, and often Python or R did not support them at all.

3

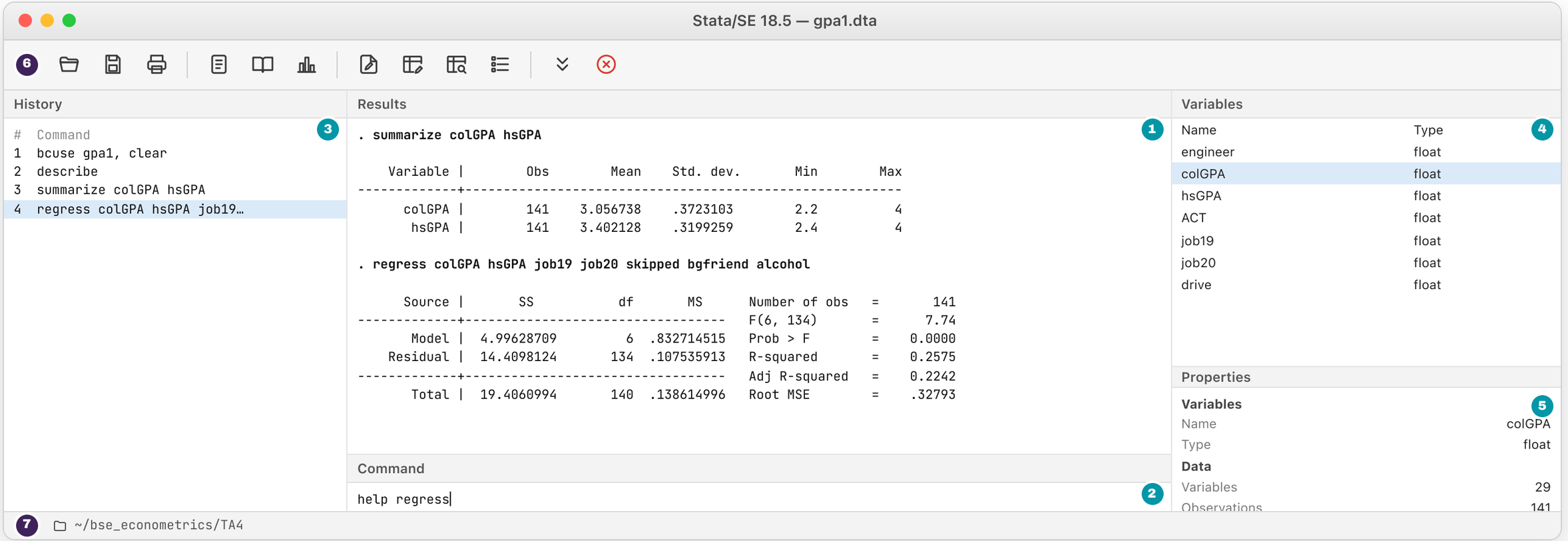
Today

Most of those methods now exist in Python, and AI tools make porting a new one easy. Python adds **speed**, better **data manipulation**, and it is **open source**.

So why learn it?

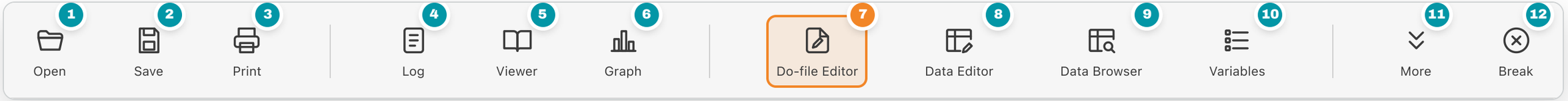
A large share of published **replication packages** are do-files, and many research institutes, central banks and public agencies still work in Stata. You need to be able to **read and run** it.

The Stata window



1. **Results**: every command and its output
2. **Command**: type one command, press Enter
3. **History**: past commands; click one to reuse it
4. **Variables**: the columns of the data in memory
5. **Properties**: details of the selected variable and dataset
6. **Toolbar**: editors, browser, log (next slide)
7. **Working directory**: where Stata reads and saves files

The toolbar



FILES

1. **Open** a dataset (`.dta`). In code: `use`
2. **Save** the data in memory. In code: `save`
3. **Print** the Results window

OUTPUT

4. **Log**: record all the output to a file. In code: `log using`
5. **Viewer**: help files. In code: `help regress`
6. **Graph**: bring the graph window to the front

WRITE CODE

7. **Do-file Editor**: where we write and run all our code. In code: `doedit`

LOOK AT THE DATA

8. **Data Editor**: spreadsheet view, cells can be changed
9. **Data Browser**: same view, **read-only**. In code: `browse`
10. **Variables Manager**: names, labels, types

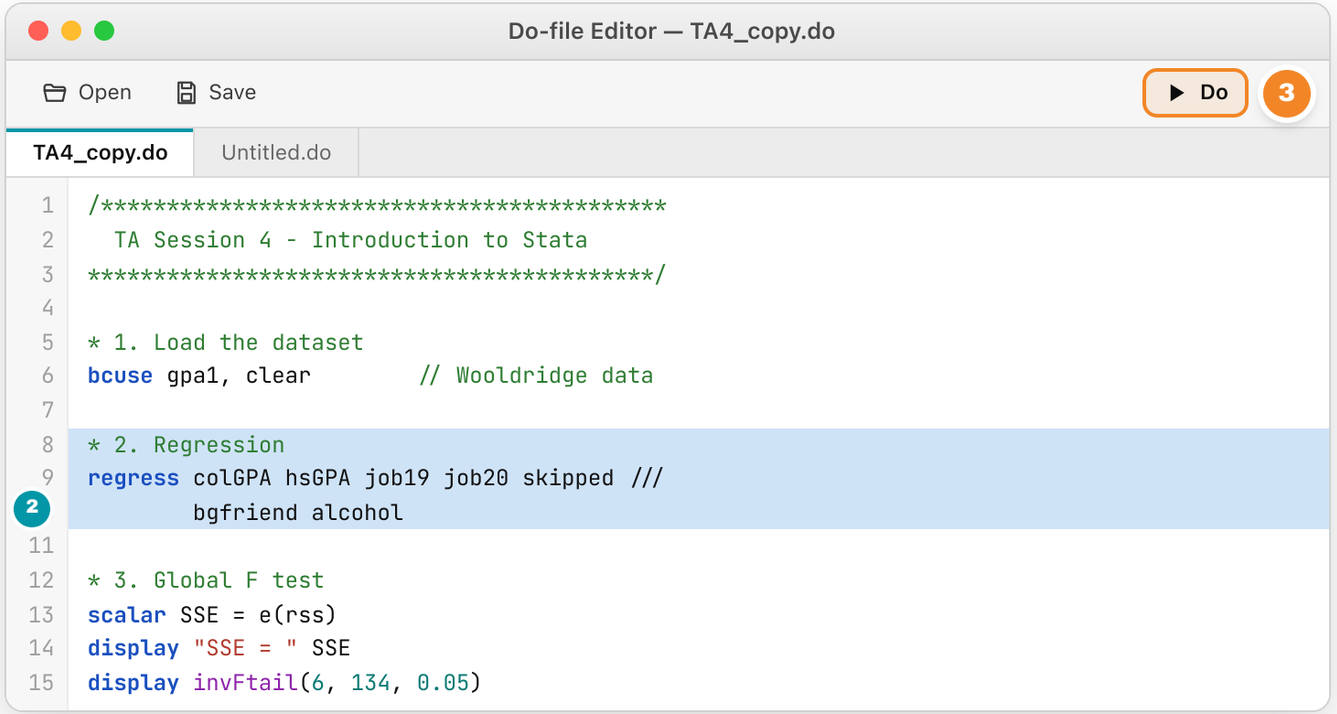
CONTROL

11. **More**: when long output pauses with `--more--` , show the next page
12. **Break**: stop the command that is running (a slow loop, a wrong command)

The Do-file Editor

1. **Open it**: toolbar button Do-file Editor, or type `doedit` in the Command window
2. **Select** the lines you want to run. Nothing selected runs the **whole** file
3. **Click Do**, or use the shortcut:

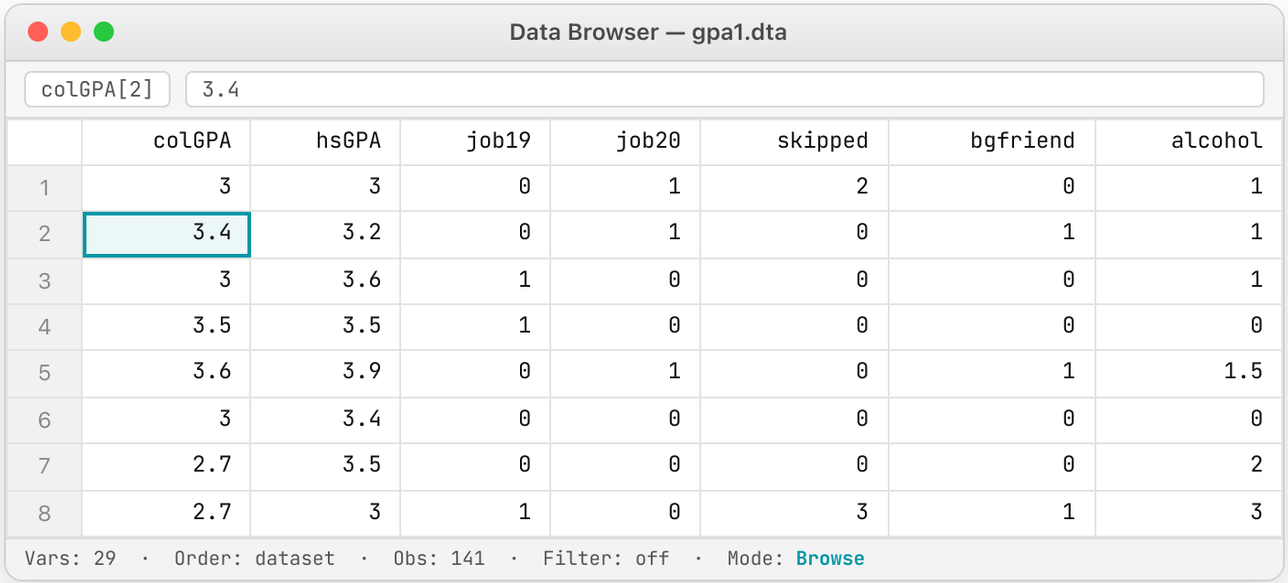
	Run (Do)
macOS	⌘ ⇧ D
Windows	Ctrl D



Always work in a do-file

The Command window is for quick checks. A do-file is a record of **everything** you did, that you (and we, when grading) can re-run.

Looking at the data: the Data Browser



	colGPA	hsGPA	job19	job20	skipped	bgfriend	alcohol
1	3	3	0	1	2	0	1
2	3.4	3.2	0	1	0	1	1
3	3	3.6	1	0	0	0	1
4	3.5	3.5	1	0	0	0	0
5	3.6	3.9	0	1	0	1	1.5
6	3	3.4	0	0	0	0	0
7	2.7	3.5	0	0	0	0	2
8	2.7	3	1	0	3	1	3

Vars: 29 · Order: dataset · Obs: 141 · Filter: off · Mode: Browse

Open it with the toolbar button or by typing `browse`.

- Each **row** is an observation (a student)
- Each **column** is a variable
- The bar on top shows the selected cell: `colGPA[2]` = 3.4

Never fix data by hand

The Data **Editor** lets you change cells, but nobody can reproduce a click. Correct values with a line in your do-file:

```
1  replace colGPA = 3.4 in 2
```

Anatomy of a Stata command

Almost every Stata command follows the same pattern (dashed parts are optional):




```
summarize colGPA hsGPA
summarize colGPA if bgfriend == 1
regress colGPA hsGPA skipped, robust
bysort bgfriend: summarize colGPA
list colGPA hsGPA in 1/5
```

Case sensitive

`colGPA` and `colgpa` are **different** names: `regress colgpa …` returns *variable colgpa not found*.

Abbreviations

`reg` = `regress`, `sum` = `summarize`, `gen` = `generate`, `di` = `display`. Use them in the Command window, write them in full in do-files.

From Python to Stata

You already know these steps from TA1–TA3: only the syntax changes.

Task	Python	Stata
Load the Wooldridge data	`df = woo.dataWoo("gpa1")`	`bcuse gpa1, clear`
Look at the data	`df.head()`	`browse` or `list in 1/5`
Variables and types	`df.info()`	`describe`
Summary statistics	`df.describe()`	`summarize`
New variable	`df["hs2"] = df.hsGPA**2`	`generate hs2 = hsGPA^2`
OLS	`smf.ols("y ~ x1 + x2", df).fit()`	`regress y x1 x2`
Joint F test	`model.f_test("x1 = 0, x2 = 0")`	`test x1 x2`
A single number	`print(model.ssr)`	`display e(rss)`

bcuse is not built into Stata

`bcuse` (the Wooldridge datasets) is a **user-written** command, so Stata does not ship with it. Install it once with `ssc install bcuse`; from then on it works in every session, like `pip install wooldridge` in Python.

Comments: a worked do-file

Do-file Editor — gpa_analysis.do

gpa_analysis.do

```
1  /*============================================================
2    Project : What drives college GPA?
3    Question: Are working hours and drinking related to college
4              GPA, holding high-school GPA fixed?
5    Data    : gpa1 (Wooldridge), 141 students
6    Author  : Lucia Sauer                Updated: 2025-10-15
7  ============================================================*/
8
9  ** 1. Load data ***************************************
10 bcuse gpa1, clear
11
12 ** 2. Prepare variables *********************************
13 * job19 and job20 leave out students without a job:
14 * they are the base category of the regression
15 generate byte nojob = (job19 == 0 & job20 == 0)
16 tabulate nojob                // 59 students do not work
17
18 ** 3. Estimate the model *********************************
19 regress colGPA hsGPA job19 job20 skipped ///
20         bgfriend alcohol          // nojob left out on purpose
```

1. **Header** `/* … */`: what the file does, the research question, the data, who and when
2. **Section banners** `***`: the same banner for every block, so the file is easy to scan
3. **Whole-line** `*`: explain **why**, not what. The code already says what it does
4. **Inline** `//`: a short note after a command
5. **Line break** `///`: continue a long command on the next line (keep lines under 100 characters)

Style adapted from the Gentzkow–Shapiro lab manual.

Stored results: `r()` and `e()`

After every command Stata **stores** its results, so we can reuse them without copying numbers by hand.

`r()` : general commands, such as `summarize`

Results

```
. summarize colGPA
. return list

scalars:
                  r(N) =  141
               r(mean) =  3.05673759541613
                 r(sd) =  .3723103491289332
                r(min) =  2.200000047683716
                r(max) =  4
                r(sum) =  431.0000009536743
```

`e()` : estimation commands, such as `regress`

Results

```
. regress colGPA hsGPA job19 job20 …
. ereturn list

scalars:
                  e(N) =  141
               e(df_m) =  6
               e(df_r) =  134
                  e(F) =  7.743594584831989
                 e(r2) =  .2574596250621445
                e(rss) =  14.40981236138072
```

```
1  scalar SSE = e(rss)          // store a result in a scalar
2  display "SSE = " SSE         // display works as a calculator too
```

They are overwritten

The next `summarize` replaces `r()`, the next `regress` replaces `e()`. Save what you need in a `scalar` straight away.

Global Hypothesis Testing

The model

$$\texttt{colGPA}_i = \beta_1 + \beta_2 \texttt{hsGPA}_i + \beta_3 \texttt{job19}_i + \beta_4 \texttt{job20}_i + \beta_5 \texttt{skipped}_i + \beta_6 \texttt{bgfriend}_i + \beta_7 \texttt{alcohol}_i + \varepsilon_i$$

where:

- `colGPA`: college GPA
- `hsGPA`: high school GPA
- `job19`: $= 1$ if the student works ≤ 19 hours a week
- `job20`: $= 1$ if the student works ≥ 20 hours a week
- `skipped`: average lectures missed per week
- `bgfriend`: $= 1$ if the student has a boyfriend/girlfriend
- `alcohol`: average number of days per week the student drinks alcohol

`job19` and `job20`: a dummy trap?

Results

```
. tabulate job19 job20

           |         job20
     job19 |         0          1 |     Total
-----------+----------------------+----------
         0 |        59         24 |        83
         1 |        58          0 |        58
-----------+----------------------+----------
     Total |       117         24 |       141
```

Question

When `job19` = 0, is `job20` always 1? Are the two dummies perfectly collinear with the constant?

No. There are **three** groups: no job (59), ≤ 19 hours (58), ≥ 20 hours (24).

The 59 students with `job19 = job20 = 0` are the **base category**: β_3 and β_4 compare each group of workers with students who do not work.

The trap appears only if we add a third dummy `nojob`: then `nojob + job19 + job20 = 1` = the constant.

Regression output in Stata

```
1  regress colGPA hsGPA job19 job20 skipped bgfriend alcohol
```

Results

```
      Source |       SS           df       MS      Number of obs   =       141
-------------+----------------------------------   F(6, 134)       =      7.74
       Model |  4.99628709         6  .832714515   Prob > F        =    0.0000
    Residual |  14.4098124       134  .107535913   R-squared       =    0.2575
-------------+----------------------------------   Adj R-squared   =    0.2242
       Total |  19.4060994       140  .138614996   Root MSE        =    .32793

------------------------------------------------------------------------------
      colGPA | Coefficient  Std. err.      t    P>|t|     [95% conf. interval]
-------------+----------------------------------------------------------------
       hsGPA |   .4649713   .0874578     5.32   0.000     .2919951    .6379475
       job19 |  -.0052822   .0644549    -0.08   0.935    -.1327628    .1221984
       job20 |  -.0765349   .0822574    -0.93   0.354    -.2392258    .0861559
     skipped |  -.0940715   .0272287    -3.45   0.001    -.1479251   -.0402179
    bgfriend |   .0991959   .0561251     1.77   0.079    -.0118099    .2102016
     alcohol |   .0356602   .0228506     1.56   0.121    -.0095344    .0808547
       _cons |   1.476362   .3072519     4.81   0.000     .8686709    2.084052
------------------------------------------------------------------------------
```

1. **Sums of squares**: Total = Model + Residual. Residual SS is our SSE
2. **F and Prob > F**: a test of the whole model. Today's topic
3. **R^2** = Model SS / Total SS; adjusted R^2 penalises extra regressors
4. **Root MSE** = $\hat{\sigma} = \sqrt{SSE/(n-K)}$
5. **Coefficient**: the estimates $\hat{\beta_k}$
6. **Inference** on each β_k: std. error, t, p-value and 95% CI (TA3)

Does the model explain anything?

We want to test whether our regression model adds explanatory power **beyond the mean** of `colGPA`.

Exercise

1. State the **null and alternative hypotheses**.
2. Write the F**-test statistic** and its distribution under H_0.
3. Run the **restricted model**; compute the RSSE (restricted), SSE (unrestricted) and the F-statistic.
4. Find the **critical value** of the F-distribution and compute the p**-value**.
5. **Draw** the p-value and the critical value.

1–2. Hypotheses and test statistic

3. RSSE and SSE in Stata

CODE

Do-file Editor — TA4_copy.do

```
1  * Unrestricted model: SSE
2  regress colGPA hsGPA job19 job20 skipped bgfriend alcohol
3  scalar SSE = e(rss)
4
5  * Restricted model (only a constant): the fitted value is the mean
6  summarize colGPA
7  scalar colgpa_mean = r(mean)
8  egen double rsse = total((colGPA - colgpa_mean)^2)
9  scalar RSSE = rsse[1]
10
11 * F statistic
12 scalar F_manual = ((RSSE - SSE) / 6) / (SSE / (e(N) - 7))
13 display "RSSE = " RSSE "   SSE = " SSE
14 display "Manual F = " F_manual "   Stata F = " e(F)
```

OUTPUT

Results

```
. display "RSSE = " RSSE "   SSE = " SSE
RSSE = 19.406099   SSE = 14.409812

. display "Manual F = " F_manual "   Stata F = " e(F)
Manual F = 7.7435946   Stata F = 7.7435946
```

$$F = \frac{(19.406 - 14.410)/6}{14.410/134} = \frac{0.8327}{0.1075} = 7.74$$

4. Critical value and p-value

```
1 display "Critical value (5%) = " invFtail(6, e(N) - 7, 0.05)
2 display "p-value = " Ftail(6, e(N) - 7, F_manual)
```

Results

```
Critical value (5%) = 2.1669018
p-value = 3.688e-07
```

Stata function	Returns	Here
`invFtail(q, n-K, α)`	the value c with $P(F > c) = \alpha$	$F_{crit} = 2.17$
`Ftail(q, n-K, f)`	$P(F > f)$: the p-value	$p = 3.7 \times 10^{-7}$

5. Drawing the test

Multiple Hypothesis Testing

Do working hours matter?

Consider testing whether `job19` and `job20` are **jointly** significant at $\alpha = 5\%$.

Exercise

1. State the **null and alternative hypotheses**.
2. Write the F**-test statistic** and its distribution under H_0.
3. Run the **restricted model**; compute the RSSE, SSE and the F-statistic.
4. Find the **critical value** and compute the p**-value**.
5. **Draw** the p-value and the critical value.
6. **Compare** your results with Stata's `test` command.

1–2. Hypotheses and test statistic

3–4. RSSE, SSE, F and p-value

CODE

Do-file Editor — TA4_copy.do

```
1  * 1. Restricted model
2  regress colGPA hsGPA skipped bgfriend alcohol
3  scalar sse_r = e(rss)
4
5  * 2. Unrestricted model
6  regress colGPA hsGPA job19 job20 skipped bgfriend alcohol
7  scalar sse = e(rss)
8
9  * 3. F statistic, critical value and p-value
10 scalar F = ((sse_r - sse) / 2) / (sse / e(df_r))
11 display "RSSE = " sse_r "   SSE = " sse "   F = " F
12 display "Critical value (5%) = " invFtail(2, e(df_r), 0.05)
13 display "p-value = " Ftail(2, e(df_r), F)
```

OUTPUT

Results

```
. display "RSSE = " sse_r "   SSE = " sse "   F = " F
RSSE = 14.515391   SSE = 14.409812   F = .49090027

. display "Critical value (5%) = " invFtail(2, e(df_r), 0.05)
Critical value (5%) = 3.0637149

. display "p-value = " Ftail(2, e(df_r), F)
p-value = .61317149
```

5. Drawing the test

6. The one-line version: `test`

After `regress`, `test` computes the same F-test for any list of coefficients:

```
regress colGPA hsGPA job19 job20 ///
        skipped bgfriend alcohol
test job19 job20
```

Same F and same p-value as our manual computation, without running the restricted model.

Results

```
. test job19 job20

 ( 1)  job19 = 0
 ( 2)  job20 = 0

       F(  2,   134) =    0.49
            Prob > F =    0.6132
```

Regression tables in LaTeX

Step 1: label and store the models

Do-file Editor — TA4_copy.do

```
* Run once: estout provides eststo and esttab
capture which esttab
if _rc ssc install estout

* The table shows labels, not variable names
label variable colGPA   "College GPA"
label variable hsGPA    "High-school GPA"
label variable job19    "Works $\leq$ 19 h/week"
label variable job20    "Works $\geq$ 20 h/week"
label variable skipped  "Lectures skipped / week"
label variable bgfriend "Has a partner"
label variable alcohol  "Days drinking / week"

* Estimate and store three nested models
eststo clear
eststo m1: regress colGPA hsGPA skipped
eststo m2: regress colGPA hsGPA skipped bgfriend alcohol
eststo m3: regress colGPA hsGPA job19 job20 skipped bgfriend alcohol
test job19 job20
estadd scalar p_jobs = r(p)        // joint-test p-value, stored in m3
```

1. **estout**: user-written package with `eststo` and `esttab`. Install it once
2. **label variable**: the table prints the labels, so write them as you want them in the paper (LaTeX math is allowed)
3. **eststo**: stores each regression under a name; each one becomes a column of the table
4. **estadd**: adds an extra statistic to a stored model, here the *p*-value of `test job19 job20`

Step 2: export with `esttab`

Do-file Editor — TA4_copy.do

```
esttab m1 m2 m3 using "regression_table.tex", replace ///
    booktabs label b(3) se(3) ///
    star(* 0.10 ** 0.05 *** 0.01) ///
    alignment(D{.}{.}{-1}) ///
    mgroups("Dependent variable: College GPA", pattern(1 0 0) ///
        prefix(\multicolumn{@span}{c}{) suffix(}) ///
        span erepeat(\cmidrule(lr){@span})) ///
    mtitles("Baseline" "Restricted" "Unrestricted") ///
    stats(N r2 r2_a F p_jobs, fmt(0 3 3 2 3) ///
        layout("\multicolumn{1}{c}{@}" @ @ @ @) ///
        labels("Observations" "R$^2$" "Adjusted R$^2$" ///
               "F statistic" "Jobs jointly = 0 (p-value)")) ///
    title("Determinants of college GPA\label{tab:gpa}") ///
    collabels(none) nogaps
```

1. `booktabs` : academic rules (`\toprule` , `\midrule`), no vertical lines
2. `label` : variable labels instead of names
3. `b(3) se(3)` : 3 decimals, standard errors in parentheses
4. `star()` : significance stars at 10%, 5%, 1%
5. `alignment(D…)` : numbers aligned on the decimal point (`dcolumn`)
6. `mgroups()` , `mtitles()` : header with the dependent variable and the model names
7. `stats()` : n, R^2, F and the p-value we added
8. `title(…\label{})` : caption, and a label to cite it with `\ref`

Dollar signs in Stata strings

`$name` is a global macro in Stata: `"$p$-value"` silently becomes `"-value"` . Write inline math as `\(p\)` instead.

Step 3: use it in your document

Put `regression_table.tex` next to your `main.tex` (in Overleaf: **Upload**), then:

```latex
\documentclass{article}
\usepackage{booktabs}  % \toprule, \midrule
\usepackage{dcolumn}   % decimal alignment
\begin{document}

As shown in Table~\ref{tab:gpa},
one more point of high-school GPA ...

\input{regression_table.tex}

\end{document}
```

Same workflow as `stargazer` in TA3: change the model, re-run the do-file, and the table updates itself.

As shown in Table 1, one more point of high-school GPA is associated with a college GPA about 0.46 points higher, while working hours are not jointly significant ($p = 0.613$).

Table 1: Determinants of college GPA

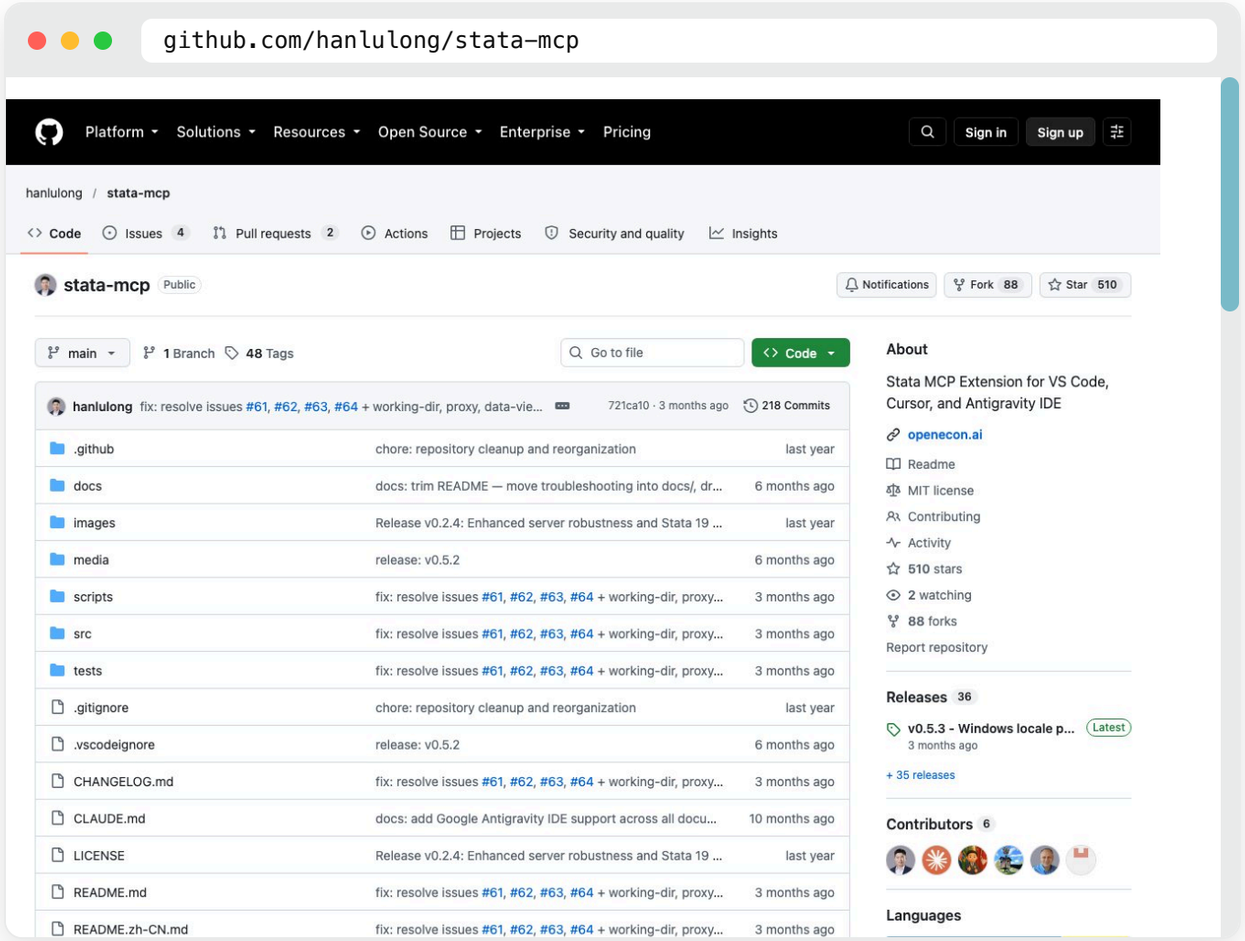
	Dependent variable: College GPA		
	(1) Baseline	(2) Restricted	(3) Unrestricted
High-school GPA	0.459***	0.467***	0.465***
	(0.088)	(0.087)	(0.087)
Lectures skipped / week	−0.077***	−0.093***	−0.094***
	(0.026)	(0.027)	(0.027)
Has a partner		0.092*	0.099*
		(0.055)	(0.056)
Days drinking / week		0.038*	0.036
		(0.021)	(0.023)
Works $\leq$ 19 h/week			−0.005
			(0.064)
Works $\geq$ 20 h/week			−0.077
			(0.082)
Constant	1.579***	1.452***	1.476***
	(0.303)	(0.305)	(0.307)
Observations	141	141	141
R^2	0.223	0.252	0.257
Adjusted R^2	0.211	0.230	0.224
F statistic	19.77	11.46	7.74
Jobs jointly = 0 (p-value)			0.613

Standard errors in parentheses

* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$

Bonus: Stata + AI 😛

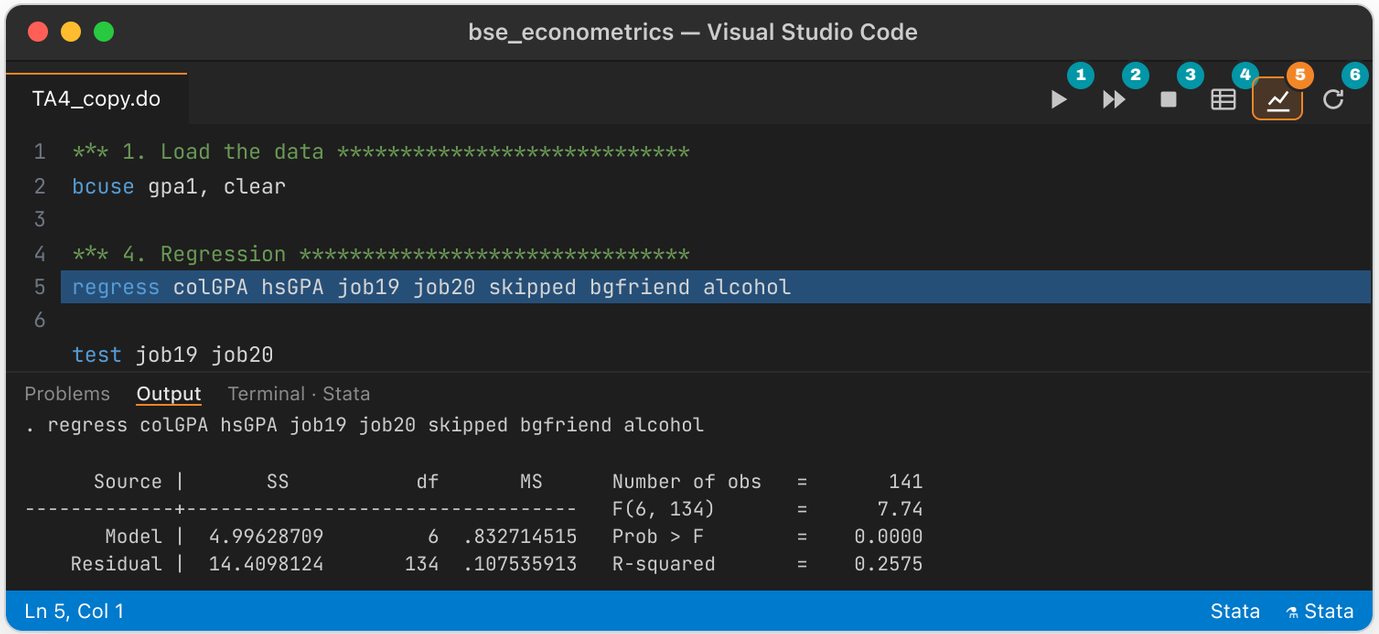
What is the Stata MCP? 😛



1. **MCP** (Model Context Protocol): an open standard that lets an AI assistant (Claude Code, Copilot, Cursor…) **use external tools**
2. **stata-mcp**: a free VS Code / Cursor extension (Stata 17+). Runs do-files from the editor, and lets the AI call `stata_run_selection` and `stata_run_file` and **read the output**
3. **Why**: Python and Stata in one editor, with one assistant for both. No copy-pasting between windows

> **Not part of the course** 😛
>
> You do **not** need it for the assignments or the exam. Fun fact: every Stata output in these slides was produced and checked this way.

Running Stata in VS Code

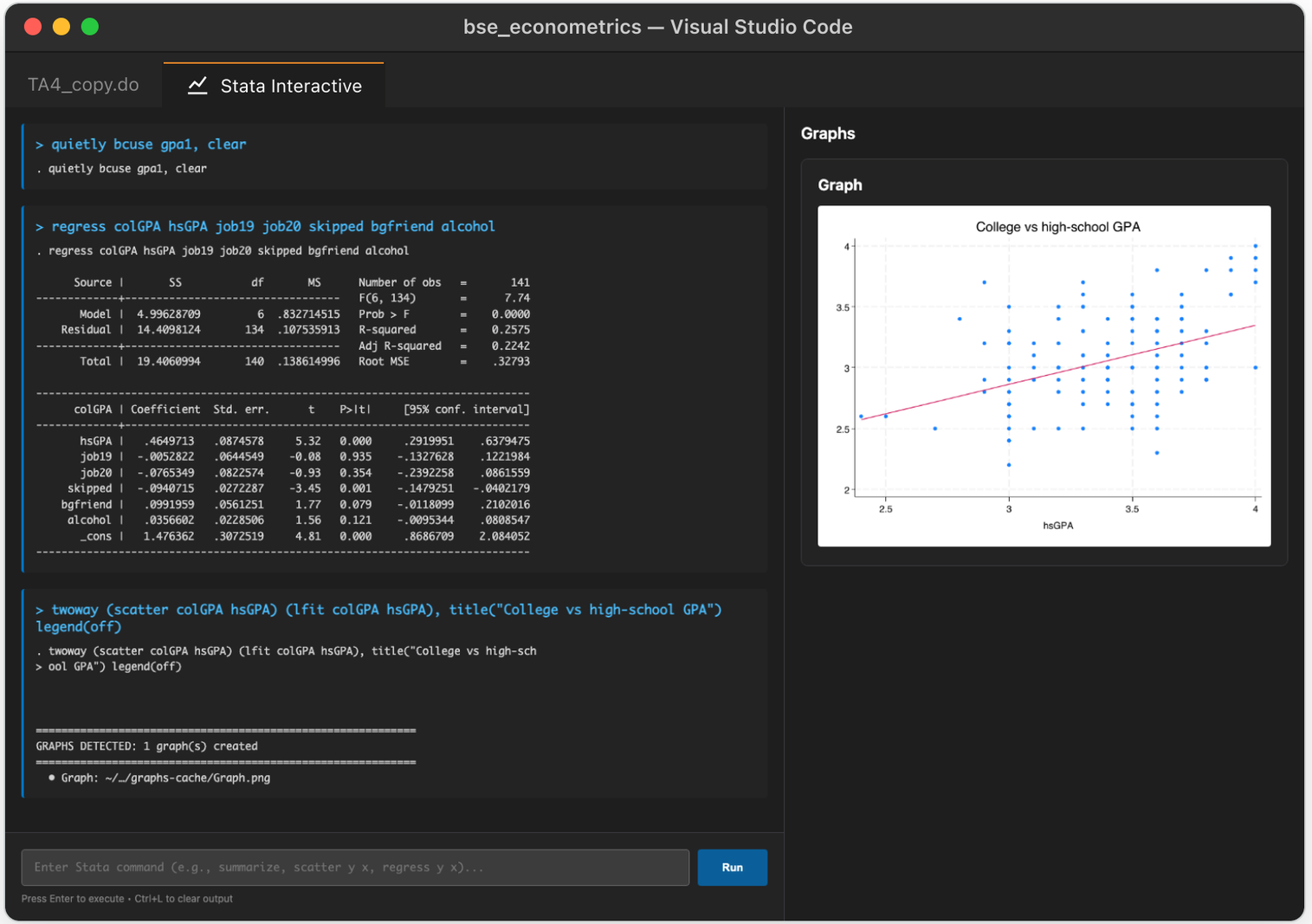


Open a `.do` file: the extension adds **six buttons** to the editor toolbar.

1. **Run selection** ⌘ ⇧ ↵
2. **Run file** ⌘ ⇧ D, as in Stata
3. **Stop** ⌘ ⇧ C, like Break
4. **View data**: a Data Browser, with `if` filters
5. **Interactive mode**: next slide
6. **Restart session**: like closing and reopening Stata

The output goes to the **Output** panel, and the status bar shows **Stata** when the server is ready. On Windows use Ctrl instead of ⌘.

Interactive mode



1. **Type a command** in the bar at the bottom and press Enter, as in Stata's Command window
2. **Output** appears on the left, one cell per command
3. **Graphs** appear on the right, next to the output

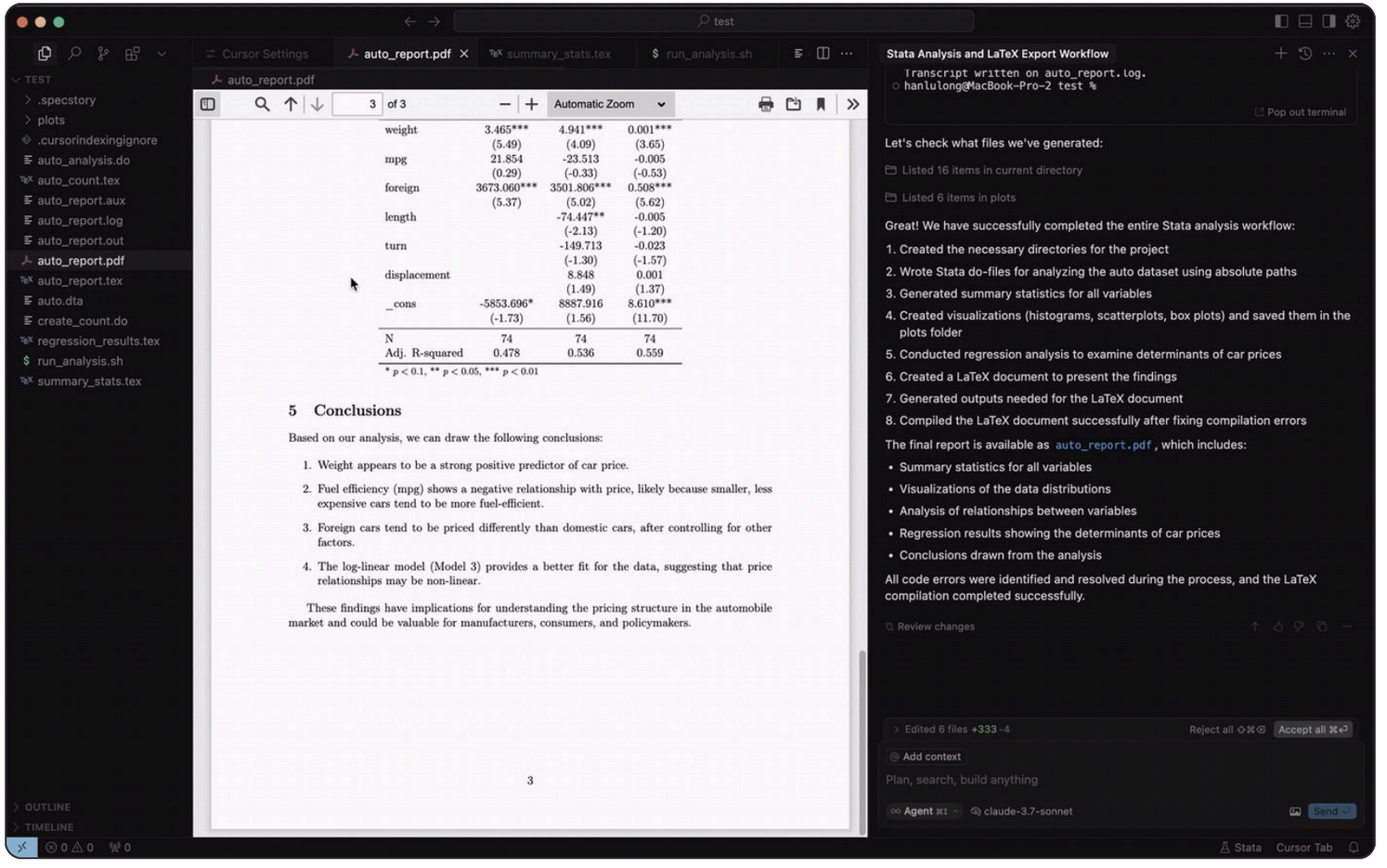
Real screenshot: the regression of this session and a scatter of `colGPA` on `hsGPA`, run in the extension's interactive window.

Or let the AI drive

1. **Ask**: describe the analysis in the chat. `stata-mcp` is listed as a tool, with `stata_run_selection` and `stata_run_file`

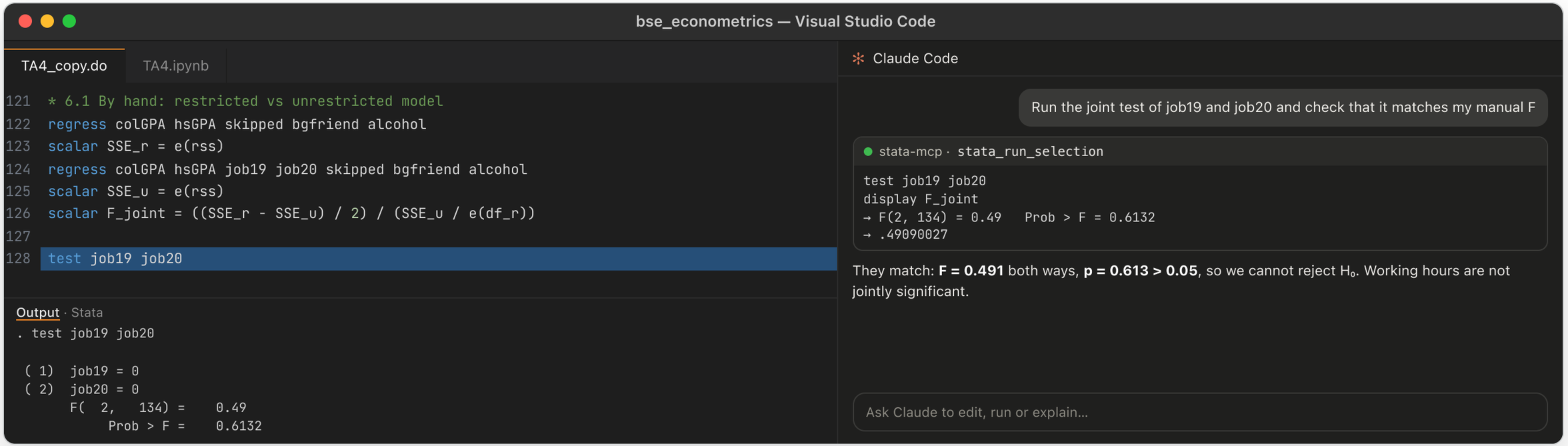
2. **Stata runs in the editor**: the AI writes the do-file, runs it, and the Stata output appears in the Output panel

3. **The result**: tables exported with `esttab`, compiled into a LaTeX report, all from the same window



Screenshots from the stata-mcp demo, github.com/hanlulong/stata-mcp (MIT licence).

With Claude Code 😛



1. **Install** the VS Code extension Stata MCP (`DeepEcon.stata-mcp`). The status bar shows Stata when it is ready

2. **Connect Claude Code**, once:
`claude mcp add --transport http stata-mcp http://localhost:4000/mcp-streamable --scope user`

Thanks!